

Speaker Notes: Capstone Presentation

Target: about 7 minutes of talking, 3 minutes for questions.

Before you start

- Fullscreen the deck.
 - Open speaker view on the second monitor if you have one.
 - Note the wall clock and write down your end time.
 - Take a breath.
-

Slide 1: Title (20s)

Open with my name, the project title, and Prof. Albuquerque as the advisor. The project is about integer optimization for logistics. Today I'm focused on the algorithm underneath it, called Branch and Bound. I built the same algorithm in three languages (Python, C++, and x86-64 Assembly) so I could separate two questions that usually get tangled up: does the algorithm matter, or does the language matter? Short answer: both, but not by the same amount.

Transition: "Let me start with the problem."

Slide 2: The 0-1 Knapsack Problem (30s)

This is the canonical knapsack. You have n items, each with a weight and a value, and a knapsack of capacity C . Pick a subset (each item is in or out, that's the "zero-one" part) that maximizes total value without going over capacity. It's NP-hard.

Why this matters for logistics: my production problem is driver-to-region assignment. Each driver is either assigned to a region or not, every assignment has a cost, and there are eligibility constraints. Same structure: binary decisions, linear objective, linear constraints. Every industrial solver I know of (CBC, Gurobi, CPLEX) uses Branch and Bound underneath.

Transition: "Let me show you a small example."

Slide 3: A Concrete Tiny Instance (30s)

Four items, capacity 8. I sorted them by value-per-weight ratio, descending. That ordering is going to matter. At $n=4$ you can list the 16 subsets by hand. But at $n=25$, which is still tiny, that's 33 million subsets. Hold this example in your head; in a couple of slides we'll walk the actual B&B tree on it.

(If asked: the optimum is B plus C, value 22, weight 8.)

Transition: "First, the simplest thing that works."

Slide 4: Baseline: Brute Force (30s)

Brute force is one loop. Iterate through every bitmask from 0 to 2^n minus 1, that's every possible subset, and keep the best feasible one. It's correct and trivial. It also scales as $O(2^n)$ time and $O(1)$ memory. Every new item doubles the work. That's the wall.

Transition: "Here's what that wall actually looks like across three languages."

Slide 5: Brute Force across Python, C + + , ASM (45s)

Same six instances ($n=3$ through $n=25$), three languages, log y-axis in seconds. Read the slope, not the heights.

Every step up in n is roughly an order of magnitude. That's 2^n in disguise. The constant factor between languages is real: at $n=25$, Assembly is about 21 times faster than Python. But the shape is the same. All three lines have the same slope.

The numbers: at $n=25$, Python takes 102 seconds, C + + takes 16, Assembly takes about 5. What happens at $n=30$? In Python that's roughly 30 minutes. $n=40$ is half a year. You can't out-engineer 2^n .

Transition: "Which is why I need a smarter algorithm. That's Branch and Bound."

Slide 6: Branch and Bound, the Idea (30s)

Three words: branch, bound, prune.

- Branch: at each item, recurse on include versus exclude. Same decision tree as brute force.
- Bound: at every node, compute an upper bound on what the subtree could possibly return.
- Prune: if that bound is no better than the best solution I already have, skip the whole subtree.

It's just informed DFS. The two clever pieces are the bound (has to be valid and cheap) and the search order. Both of those are in the next two slides.

Transition: "Here's the tree on the four-item example."

Slide 7: Decision Tree Picture (60s)

Give the audience two seconds to read.

16 possible subsets, we visit 9 nodes. Walk it left to right.

Root: nothing picked, LP bound is 27.2 (the best case if items could be fractional).

We go include-first. Include A, value 10, weight 2. Include B, value 20, weight 5. Try include

C, but C weighs 5 and we have 3 left. Infeasible (orange). Back up, exclude C, try D, same problem. Back up. We have value 20 in hand.

Now exclude A from the root. Include B, include C, value 22, weight 8. Feasible (green). That's the incumbent.

From here, look at the red nodes. Those subtrees have LP bounds at or below 22, so they can't beat what we already have. We cut them. Nine nodes touched out of 16. At $n = 25$ the ratio is closer to a thousand to one.

Key point: the reason this works is the value-per-weight sort. Include-first follows the greedy choice, so we find a strong incumbent fast, which makes the bound condition fire early. Sorting isn't cosmetic. It's what makes the pruning bite.

Transition: "Quick look at the bound itself."

Slide 8: The Bound: LP Relaxation (45s)

B&B needs a bound function. I use the LP relaxation: drop the binary constraint, let x_i be any value between 0 and 1. The fractional knapsack has a closed-form greedy answer: walk items in ratio order, take each whole until one overflows the capacity, then take a fraction of that one.

Because the items are already sorted, this is one $O(n)$ sweep, no LP solver needed. And the integer floor of the LP bound is still a valid bound for the integer problem, which is why my Assembly version stays entirely in integer registers. No floating point. Just idiv.

Transition: "Why three languages?"

Slide 9: Same Algorithm, Three Languages (30s)

Same recursion, same prune-then-branch order, same include-before-exclude descent. Python and C++ are nearly line for line. The Assembly version follows the Windows x64 ABI: r12 and r13 hold pointers to the sorted weights and values, ebx holds the running incumbent, and recursion uses call and ret on the real call stack. The bound is computed inline with integer division.

The methodological point: same algorithm in three implementations is a controlled experiment. Any timing difference now is language overhead, not algorithm difference.

Transition: "And here are the timings."

Slide 10: B&B across Python, C++, ASM (45s)

Same six instances, same three languages, same machine. Look at the y-axis. Microseconds, not seconds.

Brute force at $n = 25$ in Python was 102 seconds. B&B at $n = 25$ in Python is about 97 microseconds. The language ranking is preserved (Assembly beats C++ beats Python), but the absolute scale dropped by six orders of magnitude.

This is what I wanted to show. Algorithm and language are two independent axes. They multiply, they don't cancel each other. The algorithm win is six orders of magnitude. The language win is about one. Both real, but the algorithm dominates.

Transition: "Why does this matter for the capstone?"

Slide 11: Connection to My Capstone (30s)

Left side: the production track is a binary ILP. Drivers to regions, minimize cost, respect eligibility. I model it with PuLP, and PuLP hands it to CBC, which is a Branch and Bound solver. The algorithm I just walked through is the same family as what's actually solving my production problem. Knapsack is the didactic sibling.

Right side: the LLM track. Knapsack is a perfect testbed because it's small, well-defined, and I have ground truth from this hand-built work. When I ask an LLM to write a knapsack solver later, I can measure whether it produced B&B or just brute force.

Transition: "Here's the architecture in one picture."

Slide 12: Where B&B Fits in the Capstone (30s)

Top row: production. CSV in, preprocessing, PuLP model, CBC solver, assignments out.

Bottom row: this work. Knapsack instances feed two independent benchmark tracks, brute force on top, B&B below, three languages each.

The link in the middle: CBC and my hand-built B&B are the same algorithm family. That's why this isn't a side project. I'm studying the engine I depend on.

Transition: "Three takeaways, then questions."

Slide 13: Takeaways (30s)

Three things to leave you with.

1. Algorithm choice dominates language choice. Six orders of magnitude from B&B versus one order from Assembly. Get the algorithm right first.
2. Same algorithm in three languages is a controlled experiment. It cleanly separates the two dimensions.
3. This is the engine of the tools I'm using. PuLP, CBC, Gurobi all sit on top of Branch and Bound. Understanding knapsack means understanding what's happening when I call `prob.solve()`.

Close with: "Thank you. Happy to take questions."

Q&A: have these ready

Why not dynamic programming?

DP is pseudo-polynomial, $O(n \text{ times } C)$ in capacity. It's great when capacity is small, and I actually use it as a verification cross-check in the test suite. For the LP-to-ILP connection I wanted to study, B&B is the right algorithm because it's what CBC is doing.

Why Assembly? Isn't that overkill?

Two reasons. One: it gives me a ceiling. ASM is what you get when nothing is in your way, so it sets a clean upper bound on what hand-tuning is worth. Two: the project has a three-language comparison angle, and ASM is the most informative bottom end. Whether you'd actually maintain ASM in production is a separate question, and the answer is usually no.

What's the LP relaxation actually doing?

It's solving the fractional knapsack. Same items, but you can take a fraction of an item. That has a closed-form greedy solution when items are sorted by value-to-weight ratio. The relaxed optimum is always at least as large as the integer optimum, so it's a valid upper bound. Because the relaxation is tight, the bound prunes aggressively.

Does this scale to your production instance?

The driver-region instance is small (four drivers, three regions) and CBC solves it in milliseconds. Where this work pays off is in understanding what CBC is doing, so when I scale to a real fleet I'll know which knobs to turn: cuts, heuristics, warm starts.

How did you verify correctness across the three languages?

Every implementation returns the optimal value, and I cross-checked against three things: the brute-force baseline on small instances, PuLP/CBC as an independent reference, and a pytest suite that runs the canonical instances. All three languages return the same optima.

Why didn't you compare B&B against brute force on a single chart?

That was a deliberate choice. They're different algorithms with different asymptotic complexities, and the comparison is in some sense unfair: milliseconds against minutes. It reads cleaner to present each algorithm on its own and let the y-axis units do the comparison. That's what the two plots do: log seconds for brute force, microseconds for B&B.

Delivery tips

- Don't read the slides. The audience can read. Add what's not on the slide: the why, the intuition, the connection.
- Pause after big claims. "Six orders of magnitude" deserves a one-second pause.
- Look at the audience, not the screen. Glance at the screen only to point.
- If I blank, go to the next slide. No apology, no backtracking. Forward.
- Time check at slide 7. If I'm past 3:30, speed up the bound slide. Under 3:00, slow down the decision tree.
- End on time. Stopping at 7:00 with a minute of buffer beats rushing the takeaways.

You know this material. The job at the podium is to let the audience see that you know it.